

HOMEWORK 5

>>NAME HERE<<

>>ID HERE<<

Instructions:

Use this LaTeX file as a template to develop your homework and submit it as a single PDF file on Gradescope. Remember to assign the pages to problems when submitting.

1 Optimal Policy for Simple MDP

Consider the simple n -state MDP shown in Figure 1. Starting from state s_1 , the agent can move to the right (a_0) or left (a_1) from any state s_i . Actions are deterministic and always succeed (e.g. going left from state s_2 goes to state s_1 , and going left from state s_1 transitions to itself). Rewards are given upon taking an action from the state. Taking any action from the goal state G earns a reward of $r = +1$ and the agent stays in state G . Otherwise, each move has zero reward ($r = 0$). Assume a discount factor $\gamma < 1$.

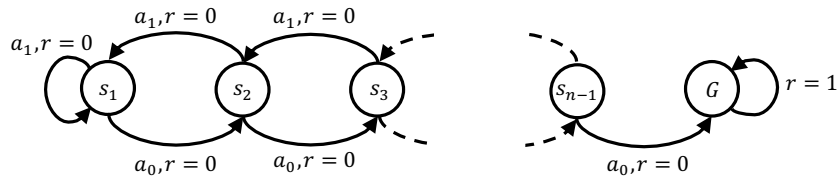


Figure 1: n -state MDP

1.1 Optimal action [5 points]

What is the optimal policy, and does it depend on the value of the discount factor γ ? Explain your answer.

[Your answer here.](#)

1.2 Optimal value function [10 points]

Find the optimal value function for all states s_i and the goal state G .

[Your answer here.](#)

1.3 Adding a constant to each reward [20 points]

Consider adding a constant c to all rewards (i.e. taking any action from states s_i has reward c and any action from the goal state G has reward $1 + c$). Can adding a constant reward c change the optimal policy? Derive the new optimal value function for all states s_i and the goal state G . Explain your answer.

[Your answer here.](#)

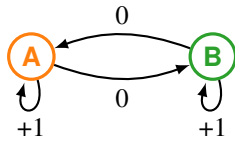
1.4 Scaling all the rewards by a constant [15 points]

After adding a constant c to all rewards now consider scaling all the rewards by a constant a (i.e. $r_{new} = a(c + r_{old})$). Can scaling rewards by a change the optimal policy? Derive the new optimal value function for all states s_i and the goal state G .

[Your answer here.](#)

2 Q-learning

Consider the following Markov Decision Process. It has two states s . It has two actions a : move and stay. The state transition is deterministic: “move” moves to the other state, while “stay” stays at the current state. The reward r is 0 for move, 1 for stay. There is a discounting factor $\gamma = 0.8$.



The reinforcement learning agent performs Q-learning. Recall the Q table has entries $Q(s, a)$. The Q table is initialized with all zeros. The agent starts in state $s_1 = A$. In any state s_t , the agent chooses the action a_t according to a behavior policy $a_t = \pi_B(s_t)$. Upon experiencing the next state and reward s_{t+1}, r_t the update is:

$$Q(s_t, a_t) \leftarrow (1 - \alpha)Q(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a'} Q(s_{t+1}, a') \right).$$

Let the step size parameter $\alpha = 0.5$.

2.1 Greedy [15 points]

Run Q-learning for 200 steps with a deterministic greedy behavior policy: at each state s_t use the best action $a_t \in \arg \max_a Q(s_t, a)$ indicated by the current Q table. If there is a tie, prefer move. Show the Q table at the end.

[Your answer here.](#)

2.2 ϵ -greedy [15 points]

Reset and repeat the above, but with an ϵ -greedy behavior policy: at each state s_t , with probability $1 - \epsilon$ choose what the current Q table says is the best action: $\arg \max_a Q(s_t, a)$; Break ties arbitrarily. Otherwise, (with probability ϵ) uniformly chooses between move and stay (move or stay both with $1/2$ probability). Use $\epsilon = 0.5$.

[Your answer here.](#)

2.3 Bellman equation [20 points]

Without doing simulation, use Bellman equation to derive the true Q table induced by the MDP.

[Your answer here.](#)